(گزارش فاز اول پروژه کامپایلر)

آرنوشا نجفی ، نیلوفر ریاحی ، نگین قربانی

گرامر(grammer) یا همان قواعد زبان ما به صورت زیر است :

GSM-> (dec | equation | Loop | If)*

dec -> " int" id (, id)* | id (, id)* = expr (, expr)*

operator -> = | += | -= | *= | /= | %=

operatorCondition -> == | != | <= | >=

equation -> id operator Expr

expr -> term ((" + " | " -") term)*

term -> factor ((" * " | " / " | " % ") factor) *

factor -> final ((" ^ ") final)*

final -> id | num | "("Expr")"

id -> [a-z AZ]+

Num -> 0 | [1-9]([0-9])*

condition -> expr operatorCondition expr

If -> "if" C : Begin (equation)* end ((Elif)* | Else)

Elif -> "elif" C : Begin (equation)* end

Else -> "else" C : Begin (equation)* end

C -> Condition ((and | or) Condition)*

Loop -> loopc C : Begin (Equation)* end

توضیحات گرامر:

GSM شروع گرامر است که میتواند به 4 حالت Loop، equation ، dec یا If برود.حال اگر مسیر ما در هر کدام از این حالت ها پیش رود به ترمینال ها و حالت های مطلوب و خواسته شده توسط رشته می رویم. در ادامه factor, term , dec , expr و final جای گیری رشته یعنی مکان id و num و همچنین operator های مورد نیاز را مشخص میکند.در ادامه condition تعریف شده است که شرط های if و elif و else و.. را چک میکند.قابل توجه است که در اینجا c رشته شرط ما را مشخص می کند و ساختار بقیه رشته هم با توجه به نوع شرط ما در هر حالت تولید می شود.

توضیحات **lexer** (تحلیل لغوی):

در فایل lexer.h در کلاس token ما انواع token ها را مشخص کردیم (مثل = ، =+ ، == ، ^ ، / و ...) و یکسری از توابع را که مربوط به token ها هست ، پیاده سازی کردیم . تابع getKind نوع token را مشخص میکند و به ما برمیگرداند . تابع getText هم خود token

را برمیگرداند . تابع isOneOf چک میکند که آیا چند تا token از جنس مشخص هست یا نه . تابع next به ما token بعدی را میدهد و تابع formToken نیز token مورد نظر را میسازد.

در فایل lexer.cpp هم تابع next را پیاده سازی میکنیم و تمام اسپیس ها را در نظر نمیگیریم و هر سری چک میکنیم که اگر به حرفی رسیده ایم ، آیا کلمات کلیدی هستند یا نه . همینطور به اعداد و عملگر ها نیز توجه می کنیم و آن ها را چک می کنیم .در نهایت token هایمان را با تابع form Token میسازیم .

توضیحات پارسر(**parser**) و تحلیل نحوی(**:AST**)

در فایل های AST و parser کلیت کار روی درخت گرامر و طی شدن رشته ما روی آن است. ابتدا در فایل AST.h ساختار درخت *AST* را توضیح می‌دهیم. در واقع Node های زیر را تعریف کرده و تقسیم بندی left و right را برای آن ها مشخص میکنیم(برای این که بعدا امکان iterate کردن روی شاخه ها(branch) ها را داشته باشند)

```
class AST; //done
class Expr; //done
class GSM; //done
class BinaryOp; //done
class Statements;
class Declaration; //done
class Final; //done
class Equation; //done
class Conditions;//done
class Condition; //done
class If; //done
class Elif; //done
class Else; //done
class Loop; //done
```

سپس در parser خود، NonTerminal ها را تک به تک تعریف کردیم.
هرکدام از NonTerminal ها شرایط نیاز برای طی کردن رشته مورد نظر را ریز به ریز فراهم می کنند.

```
GSM *parseGSM();
Expr *parseStatement();
Declaration *parseDec ();
Expr *parseOperator ();
Expr *parseOperatorCondition ();
Equation *parseEquation ();
Expr *parseExpr();
Expr  *parseTerm();
Expr *parseFactor ();
Expr *parseFinal ();
Condition *parseCondition ();
If *parseIf ();
Elif *parseElif ();
Else *parseElse ();
Conditions  *parseConditions ();
Loop *parseLoop ();
```

توضیحات **sema** (تحلیل معنایی):

در این قسمت ما semantic error هایی که ممکن است رخ دهند را بررسی میکنیم. ما کار اصلی iterate(پیمایش) را روی درختی که در فایل های قبلی ساختیم انجام می دهیم(فایل های AST و parser) و در واقع از صحت و درست بودن بخش معنایی ورودی داده شده مطمئن می شویم. برای این کار باید node هایی که ممکن است در آن ها این گونه اشتباهات رخ دهند تک به تک بررسی میکنیم(و این کار را تا فرزندان آن ها پیش می رویم)

```
virtual void visit(Expr &Node) override {};
virtual void visit(Conditions &Node) override {};
virtual void visit(Condition &Node) override {};
virtual void visit(Elif &Node) override {};
virtual void visit(If &Node) override {};
virtual void visit(Else &Node) override {};
virtual void visit(Loop &Node) override {};
```

توضیحات **code generator:**

در این قسمت ، درخت ما و نود های آن ، parse شده و برای هر کدام ،instruction مناسب تولید می شود و در نهایت کل IR ساخته می شود.